CPRO 2601 Project 2 Rational

Base Image:

I chose nginx: alpine as the base image because the application only needs a web server to serve its output, reducing the need for the user to allocate storage for Node.js tools. Nginx: alpine is also used to copy only the finished files into the final image. The disk usage for the single-stage image, which used node:24 to build the app, was 413MB; the multi-stage image, which used nginx: alpine, was 93.6MB because it only copies the finished output to the image, which means the multi-stage build reduced the storage by 319.4MB.

Service Split :

The project uses 2 services: cache and web. The web service is built from a Dockerfile, while cache is created by redis:7-alpine. The two services are separate due to having two completely different responsibilities. The web service is responsible for the application, while redis is used to compose a cache. The redis: 7-alpine image allowed me to use a pre-built caching system, instead of requiring me to create a caching system. Docker was used to bring the two services together and uses a health check and depends_on configuration to control their startup.

References:

Docker Multi-stage builds: https://docs.docker.com/build/building/multi-stage/
Docker CLI configuration: docker compose config | Docker Docs
AWS SAM CLI install guide: Install the AWS SAM CLI - AWS Serverless Application Model